

Penetration Testing

Arcane 2nd (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Code Quality	9
Audit Resources	9
Dependencies	9
Severity Definitions	10
Status Definitions	11
Audit Findings	12
Centralisation	19
Conclusion	20
Our Methodology	21
Disclaimers	23
About Hashlock	24

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Arcane team partnered with Hashlock to conduct a security audit of their code. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Arcane is a Web3 privacy-focused infrastructure project building confidential and compliance-oriented blockchain solutions for decentralized finance and cross-chain applications. The protocol focuses on enabling private transactions, shielded asset transfers, and privacy-preserving DeFi interactions through advanced cryptographic mechanisms and decentralized infrastructure. Arcane's ecosystem includes components such as private liquidity infrastructure, privacy-enhanced wallets, cross-chain interoperability mechanisms, and institutional-grade compliance tooling designed to support secure and confidential on-chain activity across multiple blockchain ecosystems.

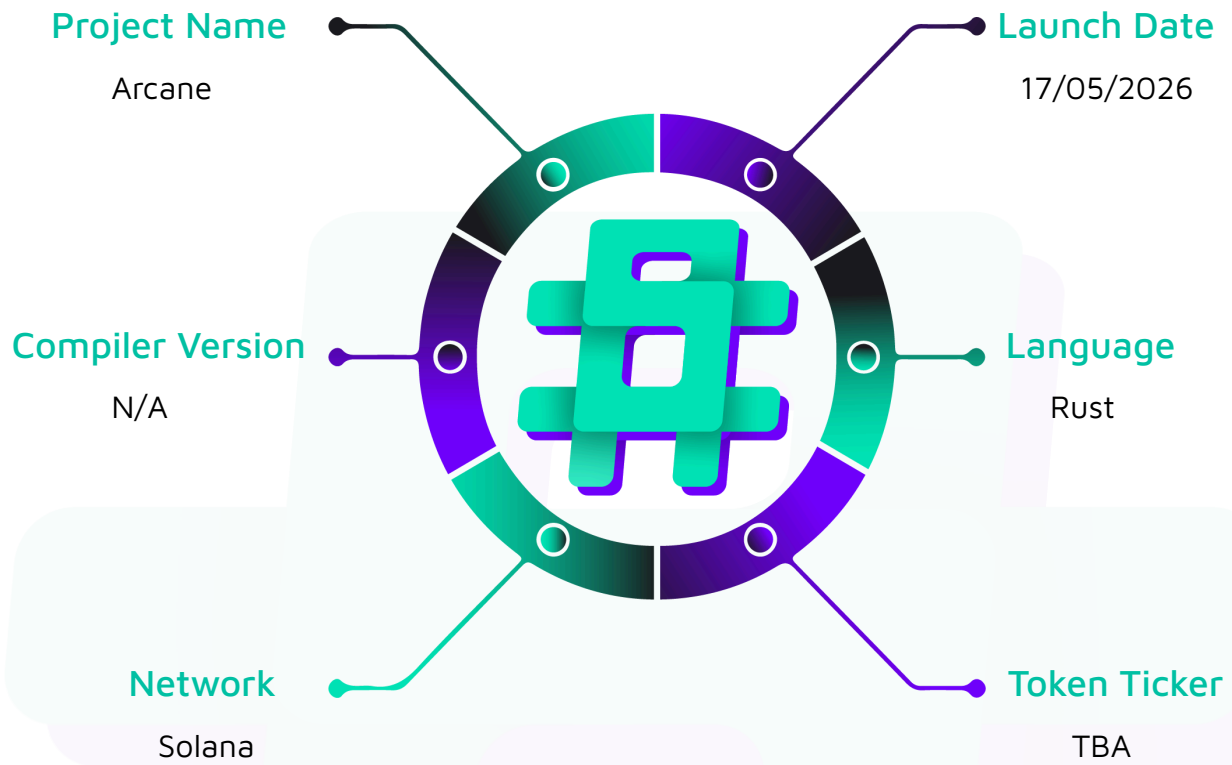
Project Name: Arcane

Project Type: DeFi

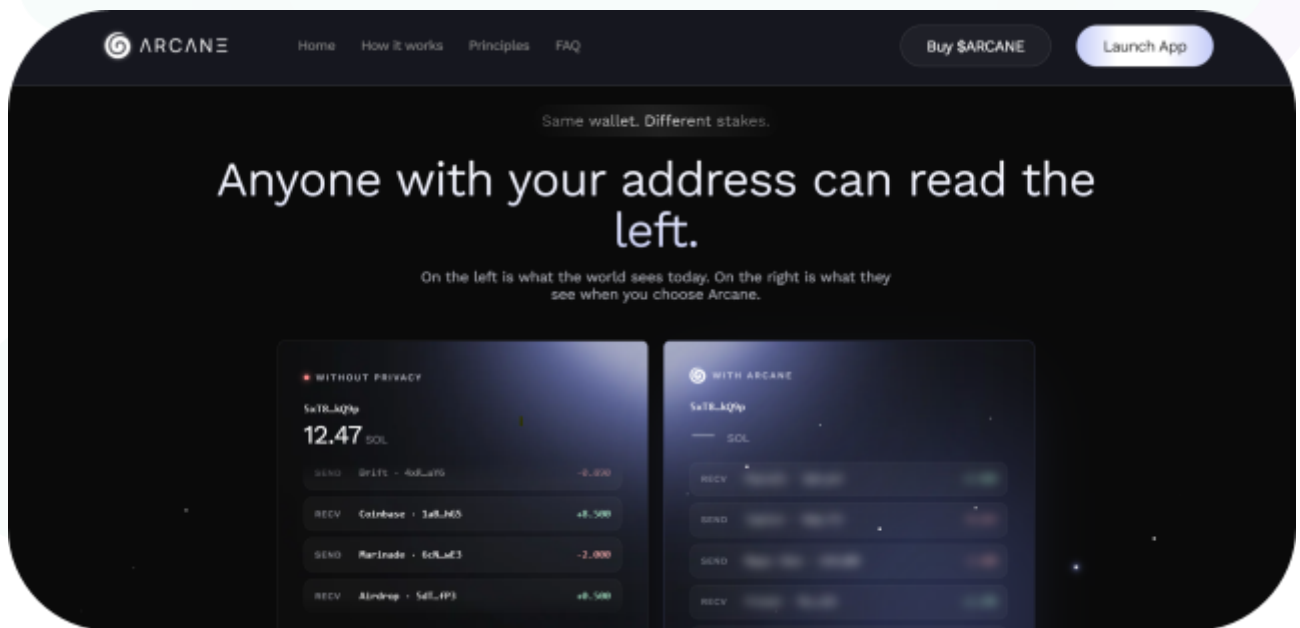
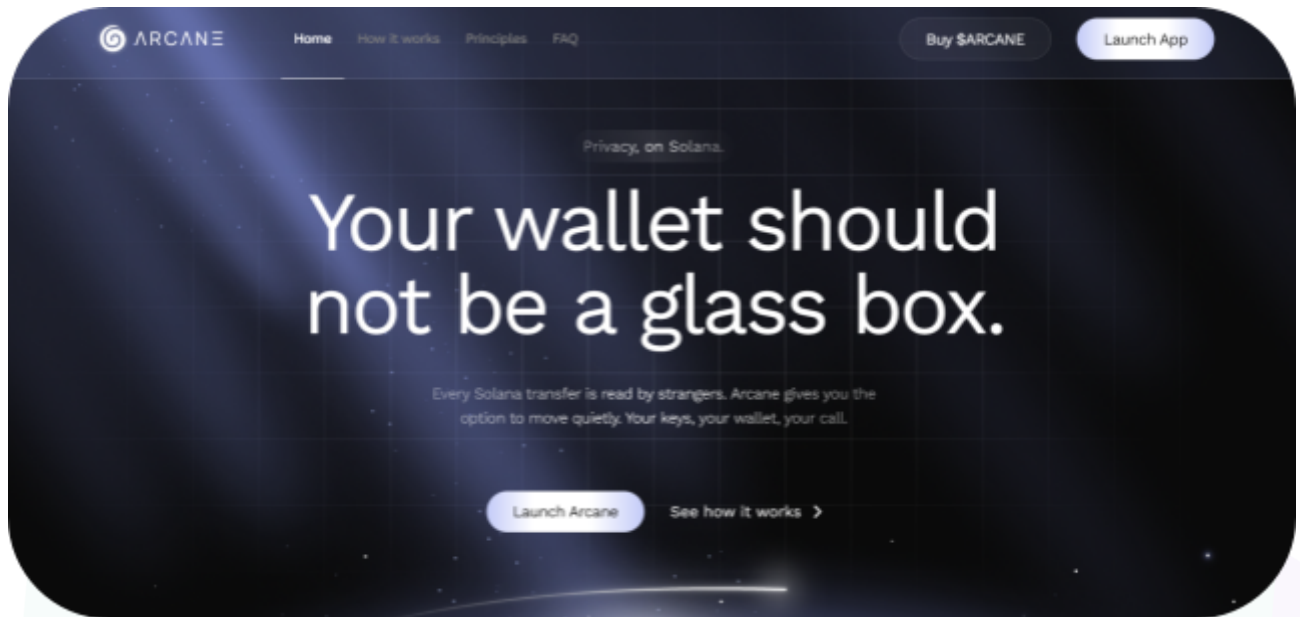
Website: <https://arcaneprivacy.com/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the Rust code within the Arcane project, the scope of work included a comprehensive review of the programs and modules listed below. We tested the programs to assess their security, functionality, and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Arcane Smart Contracts
Platform	Solana / Rust
Audit Date	July, 2026
GitHub	https://github.com/ArcanePrivacy/Relayer
Audited GitHub Commit Hash	be6fab05d2158611e2b6df145ddb85ac
Fix Review GitHub Commit Hash	8b05ba8e0f6041aae710e487184abc70b34ddc9e

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.

Security Rating

After Hashlock's Audit, we found the codebase to be **"Secure"**. The code all follows simple logic, with correct and detailed ordering. They use a series of interfaces and external dependencies, including standard open-source libraries.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Program Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 Medium severity vulnerabilities

2 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Code Quality

This audit scope involves the smart contracts of the Arcane project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Arcane project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] [arcane-relayer] src/worker.js#submitTx - relayer-funded ATA rent on the new SPL-token path is excluded from the fee-sufficiency check

Description

On the newly added SPL-token withdrawal path, the relayer keypair funds the creation of any missing Associated Token Account for the recipient, the relayer, and each fee wallet, yet the fee-sufficiency check accounts only for the signature fee and therefore never requires the user to cover that rent.

Vulnerability Details

The `submitTx` function enters the token branch whenever the request carries a `mint` field. The `createAssociatedTokenAccountInstructions` helper is called with the relayer `keypair` as the `payer`, so every account it creates debits roughly 0.00204 SOL of rent from the relayer. The subsequent check compares `platformFee` plus `relayerFeeAmount` plus the `getFeeForMessage` base fee against the user-supplied fee in `args[4]`. The `getFeeForMessage` value covers only the per-signature transaction fee and excludes lamports moved by instructions, so the Associated Token Account rent and the `args[5]` refund never enter the required-fee total. Both `mint` and the recipient in `args[2]` are attacker-controlled, so each request can target a fresh owner and mint pair whose account does not yet exist, forcing the relayer to pay rent that the collected fee does not reimburse.

```
// src/worker.js:100-105 - the relayer keypair is the ATA-creation payer
const preInstructions = await createAssociatedTokenAccountInstructions(
  keypair,
  [recipient, relayer, ...wallets.map(wallet => wallet.address)],
  mint,
  tokenProgram,
)

// src/worker.js:181-190 - sufficiency check omits ATA rent (and the refund in
args[5])
const feeResponse = await provider.connection.getFeeForMessage(message,
commitment)
const baseFee = feeResponse.value
const totalEstimatedFee = baseFee
if (
  platformFee
    .add(relayerFeeAmount)
    .add(new BN(totalEstimatedFee))
    .gt(new BN(job.data.args[4].substring(2), 'hex'))
) {
  throw new RelayerError('Provided fee is not enough. ...', 0)
}
```

Impact

Token withdrawals to not-yet-existing recipient or mint accounts repeatedly spend the relayer's SOL on unreimbursed rent, letting an economically motivated user push the relayer to a negative margin and slowly drain its operating balance, which degrades relayer availability.

Recommendation

Add the Associated Token Account rent to the required-fee total, for example by adding `getMinimumBalanceForRentExemption(AccountLayout.span)` multiplied by the number of creation instructions actually emitted, or make each recipient the payer for their own account. Reject the withdrawal unless `args[4]` covers `platformFee` plus `relayerFeeAmount` plus the base fee plus the rent and the refund.



Status

Acknowledged



QA

[Q-01] [arcane-relayer] `src/worker.js#submitTx` - unchecked null
dereference on the attacker-supplied mint account

Description

The token branch dereferences the result of `getAccountInfo(mint)` without a null check, so a well-formed but non-existent `mint` value throws a `TypeError`.

Vulnerability Details

The `mint` value originates from the `job.data.mint` field and is validated only against the `base58 addressType` regex in `src/modules/validator.js`, not for on-chain existence. When the account does not exist, `getAccountInfo` returns `null` and reading `mintAccountInfo.owner` throws. The error is caught by `processJob`, which marks the job failed and writes the raw error text to the Redis errors log surfaced by the public `/status` endpoint.

```
// src/worker.js:92-93
const mintAccountInfo = await provider.connection.getAccountInfo(mint)
const tokenProgram = mintAccountInfo.owner
```

Impact

The request fails as a job and a raw internal error string is exposed through the public `/status` endpoint, giving an attacker feedback. There is no fund loss and the worker does not crash.

Recommendation



Guard the lookup, for example `if (!mintAccountInfo) throw new RelayerError('Unknown mint')`, and confirm that `tokenProgram` is a recognised SPL token program before deriving accounts from it.

Status

Resolved

[Q-02] [arcane-relayer] src/modules/validator.js#schema - the new mint field is validated by regex only and the isAddress keyword is left unapplied

Description

The change adds `mint: addressType` to the request schema, which enforces only a base58 character and length regex, so malformed values that satisfy the regex are accepted at the HTTP boundary.

Vulnerability Details

A stricter `isAddress` custom keyword that constructs a real `PublicKey` is defined in `src/modules/validator.js` but is not referenced by any field. As a result the `mint` and `recipient` values are only pattern-checked at request time and are fully decoded later inside the worker, turning an invalid input into a queued job that fails during transaction building.

Impact

Malformed pubkeys pass request validation and become queued jobs that fail inside the worker instead of being rejected with a clean 400 response, wasting worker and RPC work.

Recommendation

Apply `isAddress: true` to `addressType`, or at least to the `mint` and `recipient` fields, so that pubkeys are fully decoded and validated at request time.

Status

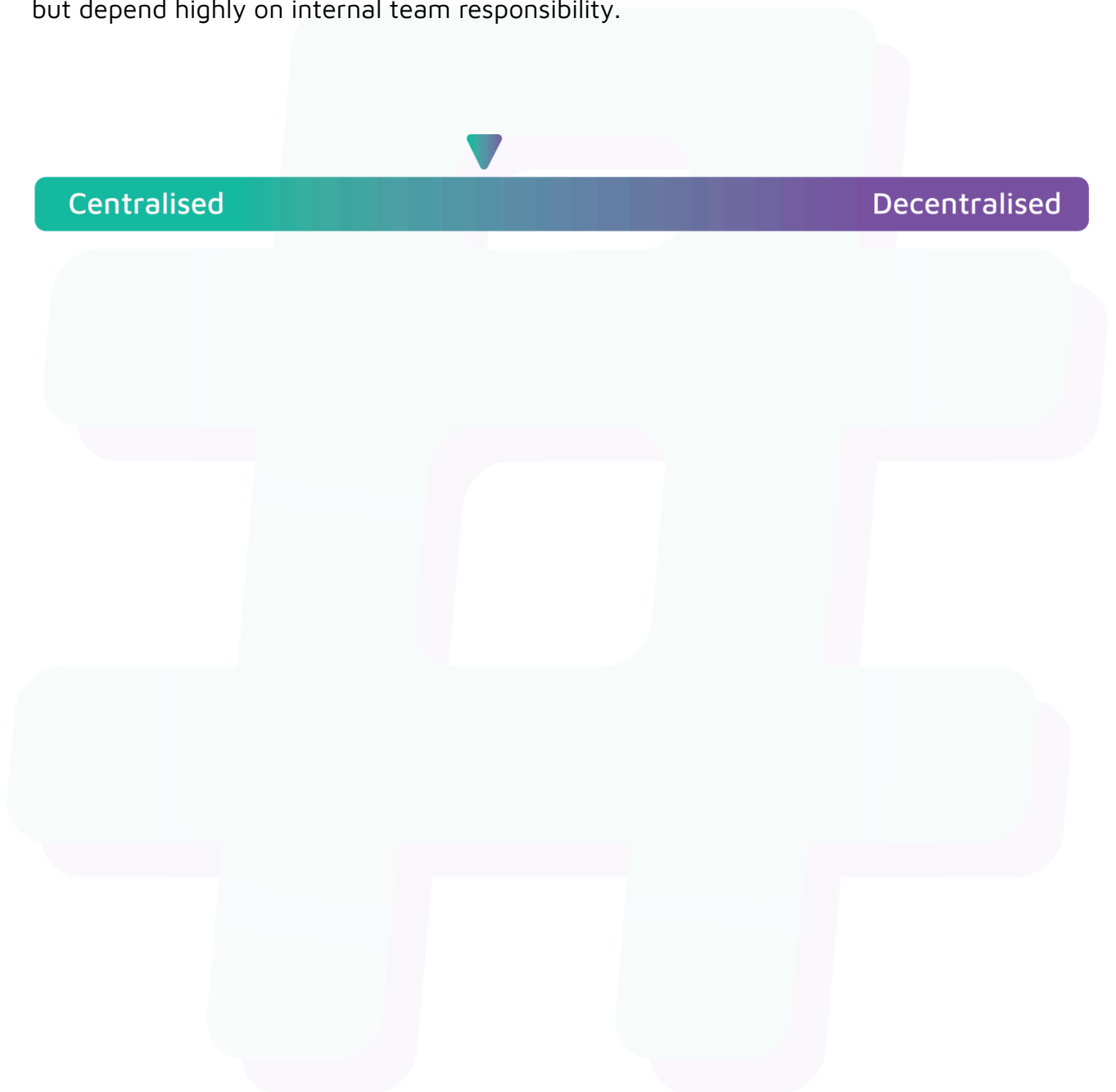
Resolved



Centralisation

The Arcane project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Conclusion

After Hashlock's analysis, the Arcane project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.